NAME: AESWARYA KRISHNAKUMAR

STUDENT ID: 22119424

loading the libraries required

```
library(rtoot)
library(tidyverse)

library(igraph)

library(lpSolve)
```

Question 1

Use the rtoot library in R to connect to the Mastodon API and download the set of toots that contain the hashtag #WSUCOMP7025. Write the code to use the downloaded data to provide a table showing each server (e.g. mastodon.social) and the count of the number of toots containing #WSUCOMP7025 provided by that server.

Authorisation and Loading the data

```
#auth_setup("mastodon.social")
```

Collecting the data

we collected all the data by using the hashtag #WSUCOMP7025 from mastodon.social with a limit of getting 100 toots.

```
toot <- get_timeline_hashtag(hashtag = "#WSUCOMP7025", instance =
"mastodon.social", limit = 100)

##   |
|                                                                      |   0%
|
|==================================                                    |  50%
|
|======================================================================| 100%
```

Working on the data

Number of rows that means total number of users.

```
nrow(toot)

## [1] 114
```

Division and extraction of URI from each toot

```
toot_uri_parts <- strsplit(toot$uri, "[^A-Za-z]+")
head(toot_uri_parts, 5)

## [[1]]
## [1] "https"         "mastodon"      "social"        "users"
## [5] "danekathrinec" "statuses"
##
## [[2]]
## [1] "https"      "mastodon"   "social"     "users"      "hishamibne"
## [6] "statuses"
##
## [[3]]
## [1] "https"        "mastodon"     "social"       "users"
"farihatasnim"
## [6] "statuses"
##
## [[4]]
## [1] "https"    "mastodon" "social"   "users"    "statuses"
##
## [[5]]
## [1] "https"    "mastodon" "social"   "users"    "thayu"    "statuses"
```

Counting the number of toots in each of these server and showed in a table

```
server_list = c()

for(i in 1:length(toot_uri_parts)){
  part_combined =
paste(toot_uri_parts[[i]][[1]],".",toot_uri_parts[[i]][[2]], collapse = "")
  part_combined = gsub(" ", "", part_combined)
  server_list = c(server_list, part_combined)
}

server_count <- table(server_list)
server_count

## server_list
##        https.convo     https.mastodon        https.mstdn
https.scholar
##                  1                102                  2
1
```

```
##       https.social         https.toot https.universeodon
##                  1                  2                  5
```

Conclusion

We extracted server names from the uri field of each toot and counted how many toots came from each server. The results show that most toots (102 out of 114) came from https.mastodon, indicating it is the main server students use to post with the hashtag #WSUCOMP7025. Other servers like https.universeodon, https.mstdn, and https.toot had only a few toots, showing lower activity.

Question 2

For each toot author obtained, download the details of the accounts that they follow and write the code to create a directed graph showing each author as a node and the edges showing who follows who. Compute the number of components in the graph and the size of each component. Plot the largest component of the graph (do your best to make it visually appealing). Comment on the structure of the graph. For the remainder of the project, we will only use the largest component of the graph.

Collect Usernames and IDs.

We started by collecting the usernames and IDs of all users who posted in the dataset.

```
all_usernames <- c()
all_user_ids <- c()

for(i in 1:nrow(toot)){
  username <- toot$account[[i]]$acct
  user_id <- toot$account[[i]]$id

  all_user_ids <- c(all_user_ids, user_id)
  all_usernames <- c(all_usernames, username)
}
```

Keep only unique users

```
unique_user_ids = unique(all_user_ids)
unique_usernames = unique(all_usernames)

unique_usernames

##  [1] "danekathrinec"                    "hishamibne20"
##  [3] "farihatasnim"                     "22109529"
##  [5] "thayu"                            "aeswarya"
##  [7] "anonuzer"                         "ahardwick"
```

```
##  [9] "Kimlong"                          "SomayehFasihi"
## [11] "19841768"                         "SenaraM"
## [13] "kamilbhura"                       "OshadaDias"
## [15] "LauroG"                           "amithlark"
## [17] "SharadaKandel"                    "ngocphuonganh"
## [19] "ngockhanhtran"                    "MahumZaidi"
## [21] "mahum"                            "kaitkayt"
## [23] "xerzat"                           "hassanbokhari"
## [25] "ayuwsu"                           "gustav2"
## [27] "chansan"                          "Thinhrichkid"
## [29] "stha_leeza"                       "wlwl"
## [31] "kiiitkaaat"                       "dofori"
## [33] "trending@mastodon.bot"            "jennyjeon"
## [35] "crayeula"                         "sauagat77"
## [37] "ankitdhakal1"                     "esha_papat"
## [39] "nimabilbil"                       "datinguyen"
## [41] "cringecompiler"                   "Islam_Merina"
## [43] "Metakilt"                         "lapark"
## [45] "22098152"                         "snisar"
## [47] "emdomingo"                        "austin_darian"
## [49] "danielsukaria"                    "austin@mstdn.party"
## [51] "asifiqbal30"                      "Remant12"
## [53] "Miya2024"                         "22105806"
## [55] "Wishawlsaif"                      "bettyndungu"
## [57] "rajanineupane"                    "vampire790"
## [59] "LeoY"                             "rajanineupane@universeodon.com"
## [61] "rajanineupane@toot.community"     "ertega"
## [63] "yiannize@universeodon.com"        "tariqfaruqi"
## [65] "ryry1999_0911"                    "jonny2k@universeodon.com"
## [67] "sonam"                            "ayazkhanP"
## [69] "nishitheda798"                    "anishbasnetworld"
## [71] "manik22@toot.community"           "manik22@universeodon.com"
## [73] "manik22@vivaldi.net"              "HM69"
## [75] "Deltanoob"                        "mkst_2367"
## [77] "manik22"                          "athiracs"
## [79] "harikrizhnavarma"                 "BibekCh"
## [81] "JessiaKyaw"                       "22131755"
## [83] "A_user"                           "HM69@mstdn.party"
## [85] "iamseemco"                        "bhubanpun"
## [87] "22105983"                         "riya_john"
## [89] "chowdhuryshaheb66"                "17979535meow"
## [91] "Ideree"                           "mgrishma17"
## [93] "6sahani"                          "22049113@universeodon.com"
## [95] "sushila@convo.casa"               "nicole05"
## [97] "lapark@scholar.social"
```

Get the list of Followed Users.

We fetched the list of users each person follows, including their IDs and usernames.

```
following_user_ids = c()
following_usernames = c()

for (i in unique_user_ids){
  people_they_follow_ids = get_account_following(i)$id
  people_they_follow_usernames = get_account_following(i)$acct


  following_user_ids = c(following_user_ids, people_they_follow_ids)
  following_usernames = c(following_usernames, people_they_follow_usernames)
}
```

Merge and Remove Duplicates

We combined original and followed users, then removed duplicates again.

```
everyone_user_ids <- c(following_user_ids, unique_user_ids)
final_user_ids <- unique(everyone_user_ids)

everyone_usernames <- c(following_usernames, unique_usernames)
final_usernames <- unique(everyone_usernames)
```

Create the Adjacency Matrix

We created a square matrix of 0s where both rows and columns represent users. This will help build the graph.

```
connection_matrix <- matrix(0, nrow = length(final_user_ids), ncol =
length(final_usernames))
rownames(connection_matrix) <- final_user_ids
colnames(connection_matrix) <- final_user_ids
```

Fill the Matrix (Mark Connections)

We filled the matrix with 1s where a user follows another.

```
for (i in unique_user_ids){
  people_followed_ids = get_account_following(i)$id

  for(j in people_followed_ids){
    connection_matrix[i,j] = 1
  }
}

rownames(connection_matrix) <- final_usernames
colnames(connection_matrix) <- final_usernames
```

Plot the Full Graph

We created the graph from the matrix and plotted it.

```
full_user_graph <- graph.adjacency(connection_matrix)
plot(
  full_user_graph,
  vertex.size = 5,
  vertex.label.cex = 0.5,
  vertex.label.dist = 0.5,
  edge.arrow.size = 0.2,
  layout = layout_with_fr
)
```

Finding the total number of components

Total number of components : 7

Calculating the size of each component

Size of the components : 154, 2, 1, 1, 1, 1, 1

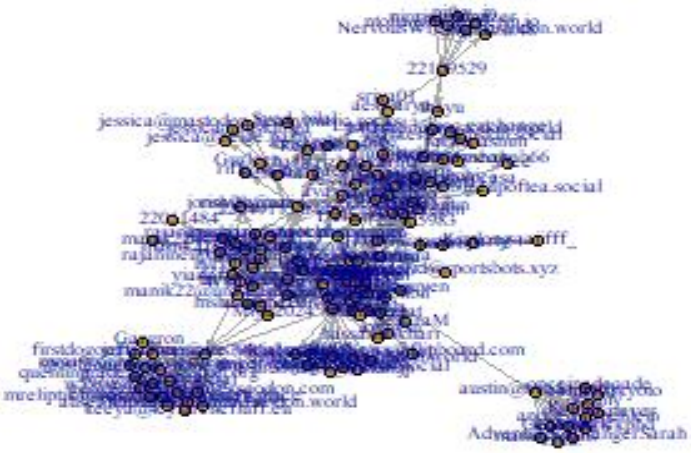
Extract and Plot the Largest Component

We focused only on the largest group of users who are all connected.

```
biggest_group_number <- which.max(components(full_user_graph)$csize)
users_in_biggest_group <- which(components(full_user_graph)$membership ==
biggest_group_number)

biggest_group_graph <- induced_subgraph(full_user_graph,
users_in_biggest_group)

plot(
  biggest_group_graph,
  vertex.size = 5,
  vertex.label.cex = 0.5,
  vertex.label.dist = 0.5,
  edge.arrow.size = 0.2,
  layout = layout_with_fr
)
```



Structure of the graph

From the final graph showing the largest component, we can observe that the structure is not random. Instead, it forms small clusters where users are grouped closely together. This suggests that users tend to follow people within their own circle, likely based on shared interests or social groups.

We also notice that some users act as central points, meaning many others follow them. These users are likely more influential or active in the network. At the same time, a few users have only one or two connections, showing limited interaction.

The graph shows a directed structure, meaning that just because one user follows another, it doesn't mean they are followed back. Overall, the structure reveals a real world social pattern people connect in groups, with a few central or well connected users bridging those groups.

Conclusion

From the output, we can see that not all users are connected some people don't follow anyone, and no one follows them either, so they end up alone or in small separate groups. But the largest group in the graph shows users who are all linked and active. This group is

the most engaged part of the network. We can use this group for deeper analysis because it shows how most users interact with each other. Overall, the directed graph helped us clearly see who follows who and how users are connected on the platform.

Question 3

Compute and report the diameter and density of the graph. Plot the in-degree distribution of the graph and estimate the Power Law coefficient (c) from the in-degree distribution. Briefly explain what the value of this coefficient reveals about the graph.

Calculate the Diameter of the Graph

The diameter represents the longest distance between any two nodes in the graph.

```
graph_diameter <- diameter(biggest_group_graph)
graph_diameter

## [1] 9
```

Calculate the Graph Density

The density tells us how tightly connected the graph is.

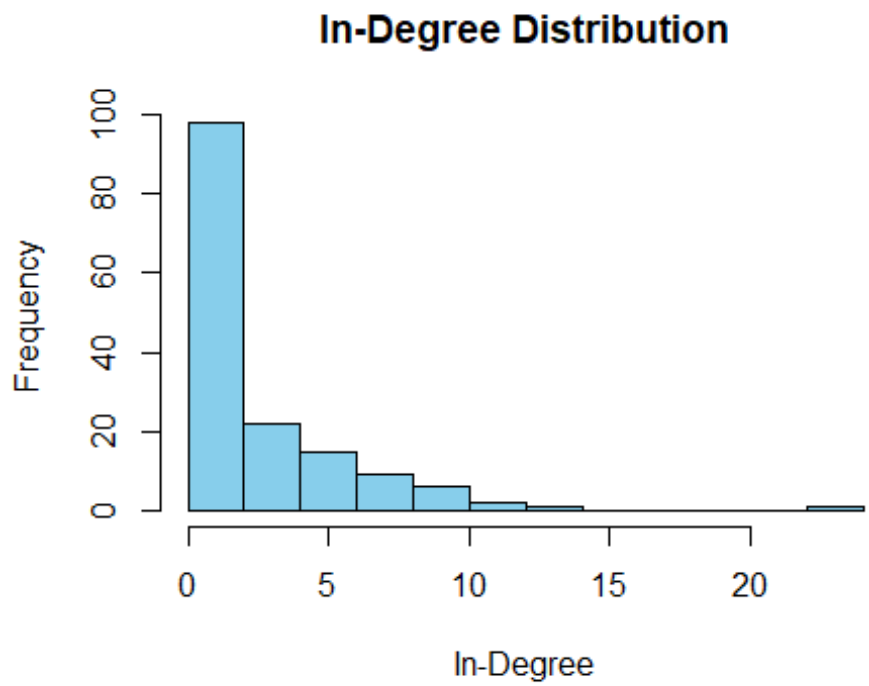
```
graph_density <- edge_density(biggest_group_graph)
graph_density

## [1] 0.01897122
```

Plot the In-Degree Distribution

We calculate how many users each user is followed by (in-degree), then plot a histogram

```
in_degrees <- degree(biggest_group_graph, mode = "in")
hist(in_degrees, col = "skyblue", main = "In-Degree Distribution", xlab =
"In-Degree")
```



Convert to Probability Distribution

To estimate the Power Law coefficient, we convert the in-degree counts to probabilities

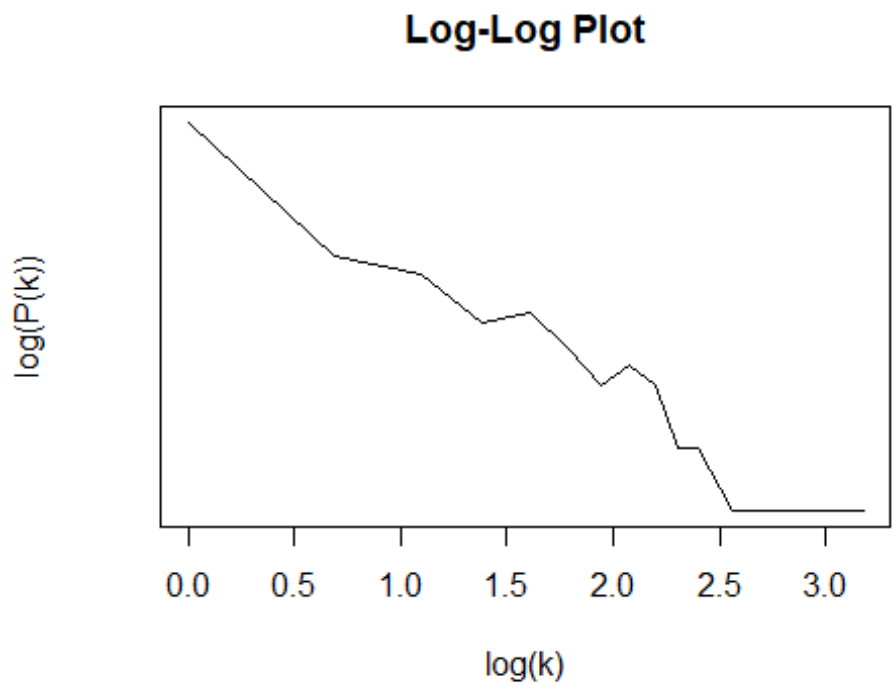
```
in_degree_table <- table(in_degrees)
prob_dist <- in_degree_table / sum(in_degree_table)
k_values <- as.numeric(names(prob_dist))
```

Filter out any zero or invalid values to avoid log(0)

```
valid_idx <- which(prob_dist > 0 & k_values > 0)
log_k <- log(k_values[valid_idx])
log_p <- log(prob_dist[valid_idx])
```

Plot the Log-Log Distribution

```
plot(log_k, log_p, type = "l", main = "Log-Log Plot", xlab = "log(k)", ylab =
"log(P(k))")
```



Estimate the Power Law Coefficient

We fit a linear model to estimate the slope (which is the coefficient c).

```
power_law_model <- lm(log_p ~ log_k)
summary(power_law_model)

##
## Call:
## lm(formula = log_p ~ log_k)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.58425 -0.14239  0.01752  0.26758  0.35064
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -0.8890     0.2004  -4.435    0.001 **
## log_k        -1.3894     0.1021 -13.605 3.17e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.298 on 11 degrees of freedom
## Multiple R-squared:  0.9439, Adjusted R-squared:  0.9388
## F-statistic: 185.1 on 1 and 11 DF,  p-value: 3.171e-08
```

Extract the Coefficient

```
power_law_coeff <- power_law_model$coefficients[2]
power_law_coeff

##     log_k
## -1.389372
```

Conclusion

From the log-log regression, we found the Power Law coefficient to be approximately –1.39.

This means that the graph follows a scale-free structure, where most users have very few followers, and only a small number of users have many followers. These few highly connected users act like influential hubs in the network.

The negative slope also tells us that the probability of a user having more followers drops quickly — the more connections a user has, the less common it is.

The high R-squared value (0.935) shows that the power law model fits the in-degree data very well.

Question 4

The Department of Education believe that people who have lunch in the same location will have similar preferences. Use the data provided with the project specification to determine if the subgraph of people exhibits Homophily.

Load and Prepare Lunch Data

We start by loading the CSV file that contains usernames and their lunch locations.

```
lunch_info <- read.csv("lunch_location.csv")
head(lunch_info)

##       Username Location
## 1   kiiitkaaat     East
## 2       dofori     East
## 3    jennyjeon     East
## 4     crayeula     East
## 5    sauagat77     East
## 6 ankitdhakal1     East
```

Match Locations to Graph Users

We assign lunch locations to users in the largest component graph (biggest_group_graph) using their usernames.

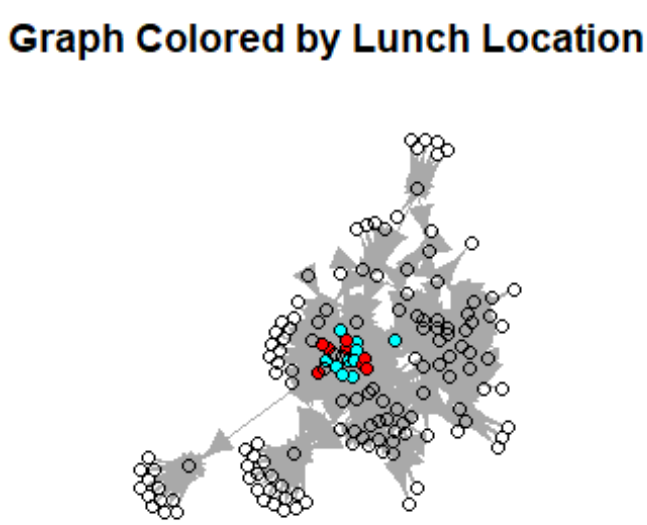
```
V(biggest_group_graph)$location <-
lunch_info$Location[match(V(biggest_group_graph)$name, lunch_info$Username)]
```

Check for Homophily Visually

We color the nodes in the graph based on lunch location.

```
location_factor <- as.factor(V(biggest_group_graph)$location)
location_palette <- rainbow(length(levels(location_factor)))  # Or use:
c("red", "blue", "green", "orange")
vertex_colors <- location_palette[location_factor]

plot(biggest_group_graph,
     vertex.color = vertex_colors,
     vertex.label = NA,
     vertex.size = 8,
     main = "Graph Colored by Lunch Location")
```



Quantify Homophily

We use assortativity to check if users are more likely connected to others with the same location. Assortativity measures how much nodes connect to others with the same attribute.

```
# Get the raw location data
location_raw <- V(biggest_group_graph)$location

# Find users who have lunch location info
valid_indices <- which(!is.na(location_raw))

# Convert valid locations to numeric codes
location_numeric <- as.numeric(as.factor(location_raw[valid_indices]))

# Create a subgraph with only those users
graph_with_location <- induced_subgraph(biggest_group_graph, vids =
valid_indices)

# Calculate assortativity (homophily score)
homophily_score <- assortativity_nominal(graph_with_location,
location_numeric, directed = TRUE)
homophily_score

## [1] -0.1886792
```

conclusion

From the output, the homophily score is -0.1886792.

If this value is close to 1, it means users strongly prefer to connect with others who eat lunch at the same place — strong homophily. If it's close to 0, it means there is no special pattern — users connect randomly across locations. The homophily score is approximately –0.19, which means that users in the graph are not likely to follow others from the same lunch location. In fact, it suggests a slight tendency to follow people from different locations. Based on this result, we can say that the graph does not show homophily based on lunch location.

Question 5

The popularity of each Mastodon account can be measured using PageRank. Measure the popularity of each Mastodon account using the Scaled PageRank algorithm, with $\alpha = 0.85$. Report the ten most popular accounts and their PageRank score, and compare the results to the in-degree of each vertex.

Create the Adjacency Matrix

We first create an adjacency matrix from the largest graph component.

```
adj_matrix <- as.matrix(as_adjacency_matrix(biggest_group_graph))
```

Avoid NaN/Zero Errors

We add a small value to avoid division by zero issues in the next step.

```
adj_matrix <- adj_matrix + 1e-5
```

Build the Probability Matrix for Scaled PageRank

```
prob_matrix <- adj_matrix %*% diag(1 / colSums(adj_matrix))

n <- nrow(prob_matrix)
alpha <- 0.85
random_jump <- matrix(1/n, n, n)

pagerank_matrix <- alpha * prob_matrix + (1 - alpha) * random_jump
```

Run the PageRank Algorithm

```
pagerank_values <- rep(1/n, n)

for (i in 1:10000) {
  pagerank_values <- pagerank_matrix %*% pagerank_values
}
```

Show Top 10 Accounts by PageRank

```
pagerank_df <- data.frame(name = V(biggest_group_graph)$name,
                          pagerank = pagerank_values)
top_pagerank <- pagerank_df %>% arrange(desc(pagerank)) %>% head(10)
top_pagerank

##                        name   pagerank
## xerzat               xerzat 0.07782436
## wlwl                   wlwl 0.06899620
## nimabilbil       nimabilbil 0.05180471
## sauagat77         sauagat77 0.04857068
## austin_darian austin_darian 0.03585630
## chansan             chansan 0.03498738
## gustav2             gustav2 0.03423037
## jennyjeon         jennyjeon 0.02377172
## snisar               snisar 0.02288162
## SharadaKandel SharadaKandel 0.02182690
```

Compare with In-Degree Rankings

```
indegree_df <- data.frame(name = V(biggest_group_graph)$name,
                          indegree = degree(biggest_group_graph, mode =
"in"))
top_indegree <- indegree_df %>% arrange(desc(indegree)) %>% head(10)
top_indegree
```

```
##                          name indegree
## lapark                 lapark       24
## Ideree                 Ideree       13
## Islam_Merina     Islam_Merina       11
## sauagat77           sauagat77       11
## wlwl                     wlwl       10
## 22098152             22098152       10
## snisar                 snisar        9
## cringecompiler cringecompiler        9
## dofori                 dofori        9
## austin_darian   austin_darian        9
```

Conclusion

From the output, we can see that the users “wlwl”, “sauagat77”, “snisar”, and “austin_darian” are present in both the PageRank and in-degree top 10 lists.

Even though the two rankings use different methods:

In-degree counts how many people follow a user

PageRank also considers who is following them (giving more weight to being followed by important people)

This overlap suggests that these users are not only followed a lot, but also have influence in the network.

In conclusion, PageRank and in-degree show different sides of popularity, but both help us identify the most important users in the network. The presence of 4 common users shows some consistency between the two methods.

Question 6

The Department of Education want to develop strategies for new students to join social groups. They want new students to visit the East and West side of the campus, but they need a strategy to maximise their chances. A set of factors were taken into account to provide the following payoff matrix. The columns show the side chosen by one new student and the rows show the side chosen by the other. The payoffs are the chance that each joins a social group.

Create the Payoff Matrix

We take the average of the two players' chances of joining a group (as shown in the table), and build the matrix.

```
# Average payoff = (Player1 + Player2)/2
payoff <- matrix(c(
  (0.6 + 0.4)/2,  (0.2 + 0.8)/2,  # Student 1 chooses East
  (0.3 + 0.9)/2,  (0.7 + 0.4)/2   # Student 1 chooses West
), nrow = 2, byrow = TRUE)

payoff
```

```
##      [,1] [,2]
## [1,]  0.5 0.50
## [2,]  0.6 0.55
```

Check Matrix Validity

Using the method from class, we build the matrix X for linear programming.

```
X <- rbind(cbind(c(1, 1), -payoff), c(0, 1, 1))
X
```

```
##      [,1] [,2]  [,3]
## [1,]    1 -0.5 -0.50
## [2,]    1 -0.6 -0.55
## [3,]    0  1.0  1.00
```

Solve the Linear Equation

We use lpSolve to compute the optimal mixed strategy.

```
result <- lp(direction = "max",
             objective.in = c(1, 0, 0),
             const.mat = X,
             const.dir = c("<=", "<=", "=="),
             const.rhs = c(0, 0, 1))
```

Get the Probability Distribution

Extract the actual strategy for Student 1 from the result.

```
prob_dist <- result$solution[2:3]
prob_dist <- matrix(prob_dist, nrow = 1)
colnames(prob_dist) <- c("East", "West")
prob_dist
```

```
##      East West
## [1,]    0    1
```

Conclusion

From the result, we can see that the best option for a student is to always choose West. The output says there's a 100% chance to go West and 0% chance to go East if they want the highest possible chance of joining a group.This choice gives them the highest chance of joining a social group based on the payoff values.

This works well because it helps students make the safest decision even when they can't talk to each other. Choosing the West side gives them the best chance to fit into a group and avoids any risk from picking the wrong side.